

西南科技大学计算机科学与技术学院

实验报告

实验名称 Linux 环境 Shell 程序开发

实验地点 东 11c225

实验日期 2024 年 10 月 31 日

指导教师 韦勇

学生班级

学生姓名

学生学号

提交日期 2024 年 11 月 13 日

一、实验目的

任务 1

掌握 Shell 编程的基本特点。

任务 2

掌握 shell 脚本的编写

任务 3

应用 Shell 程序设计方法，能分析 Shell 程序结构；掌握 Shell 程序开发环境的使用；能根据 Shell 程序功能、安全管理方面的需求，设计 Shell 程序；通过组织或参与开源项目开发，掌握项目开发、管理的方法和技术；能口头表达项目开发情况。

二、实验设计

任务 1:

本实验旨在通过编写和执行 Shell 脚本来掌握其基本特点。首先，创建一个脚本统计 /etc/passwd 文件的行数，然后通过 wc 和 grep 命令验证结果。接着，通过重定向操作符将脚本的输出和错误信息保存到文件中，以加深对 Shell 命令行操作的理解。

任务 2:

任务（1）：

- 1、使用 vim score.sh 创建一个脚本文件，并编辑代码。
- 2、首先使用 read -p 输入要查询学生的学号
- 3、判断成绩单文件是否在目录下，若不在则返回错误。
- 4、使用 grep 函数查询并获取要查询学生的成绩信息，若查询为空则输出错误，否则输出标题以及学生各个标题的信息。
- 5、给文件添加可执行权限并执行。

任务（2）：

IS_someone=who | grep \$1``：使用 who 命令获取当前登录到系统的所有用户，并使用 grep 命令查找出与输入参数\$1 匹配的用户。将结果保存在 IS_someone 变量中。

while [-z "\$IS_someone"]：进入一个循环，只要 IS_someone 变量为空就执行循环体内的操作。

sleep 3：让脚本暂停执行 3 秒钟。

echo "watch it,\$1 is online!" | wall：当循环结束时，表示找到了与输入参数匹配的用户。通过 echo 命令输出提示信息，并通过管道将信息发送给 wall 命令，以便将消息广播到所有登录的用户。

这段脚本的作用是持续检测指定用户是否登录到系统中。如果用户登录了，脚本会输出一条提示信息，并通过广播的方式发送给所有登录用户。脚本会每隔 3 秒钟执行一次检测操作，直到找到指定用户为止。

任务 3

写出全部代码中实现的全部函数的说明，内容包括：

Function（函数名称）： // 函数名称

Description（函数功能、性能等的描述）： // 函数功能、性能等的描述

Calls（被本函数调用的函数清单）： // 被本函数调用的函数清单

Called By（调用本函数的函数清单）： // 调用本函数的函数清单

File Accessed（被访问的文件）：// 被访问的文件

File Updated（被修改的文件）： // 被修改的文件

Input（输入参数说明，包括每个参数的作用、取值说明及参数间关系。）： //

输入参数说明，包括每个参数的作用、取值说明及参数间关系。

Output（对输出参数的说明）： // 对输出参数的说明。

Return (函数返回值的说明): // 函数返回值的说明
Others (其它说明): // 其它说明三、实验记录

三、实验记录

任务 1: Shell 编程基础

1、打开命令行窗口，完成如下任务：统计/etc/passwd 文件的行数

- 使用 wc (word count) 命令计算文件中行数。wc < /etc/passwd

```
[root@master ~]# wc < /etc/passwd
26   51 1290
```

这里的 < 是输入重定向操作符，它将 /etc/passwd 文件的内容作为输入传递给 wc 命令。第一个为行数（26），第二个为单词数（51），第三个为字符数（1290）。也可以直接用 -l 参数只显示行数。所以/etc/passwd 文件有 26 行。

也可以使用 grep -n "" /etc/passwd 命令，在 /etc/passwd 文件中搜索空字符串并显示行号。

```
[root@master ~]# grep -n "" /etc/passwd
1:root:x:0:0:root:/root:/bin/bash
2:bin:x:1:1:bin:/bin:/sbin/nologin
3:daemon:x:2:2:daemon:/sbin:/sbin/nologin
4:adm:x:3:4:adm:/var/adm:/sbin/nologin
5:lp:x:4:7:lp:/var/spool/lpd:/sbin/nologin
6:sync:x:5:0:sync:/sbin:/bin/sync
7:shutdown:x:6:0:shutdown:/sbin:/sbin/shutdown
8:halt:x:7:0:halt:/sbin:/sbin/halt
9:mail:x:8:12:mail:/var/spool/mail:/sbin/nologin
10:operator:x:11:0:operator:/root:/sbin/nologin
11:games:x:12:100:games:/usr/games:/sbin/nologin
12:ftp:x:14:50:FTP User:/var/ftp:/sbin/nologin
13:nobody:x:99:99:Nobody:/:/sbin/nologin
14:systemd-network:x:192:192:systemd Network Management:/:/s
19:chrony:x:998:996:/:/var/lib/chrony:/sbin/nologin
20:polkitd:x:997:995:User for polkitd:/:/sbin/nologin
21:rtkit:x:172:172:RealtimeKit:/proc:/sbin/nologin
22:pulse:x:171:171:PulseAudio System Daemon:/var/run/pulse:/
nologin
23:colord:x:996:990:User for colord:/var/lib/colord:/sbin/no
24:gdm:x:42:42:/:/var/lib/gdm:/sbin/nologin
25:mysql:x:995:989:MySQL server:/var/lib/mysql:/bin/bash
26:apache:x:48:48:Apache:/usr/share/httpd:/sbin/nologin
```

可以看到一共 26 行。

任务 2: Shell 脚本编程

任务 (1)

- 1、使用 vim score.sh 创建一个脚本文件，并编辑代码。

```
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)
#!/bin/bash

read -p "请输入学号：" student_id
# 检查成绩单是否存在
if [ ! -f "/root/workspace/shell/data.txt" ]; then
    echo "成绩单文件不存在"
    exit 1
fi

# 获取成绩单信息
result=$(grep "$student_id" /root/workspace/shell/data.txt)

# 判断查询结果是否为空
if [ -z "$result" ]; then
    echo "未找到该学号的成绩信息"
else
    second_line=$(sed -n '2p' "/root/workspace/shell/data.txt")
    echo "$second_line"
    echo "$result"
fi

-- 插入 --
```

2、在命令行中运行命令（chmod a+x score.sh）来赋予脚本执行权限：

```
Terminal 终端 ~ root@master:~
文件(F) 编辑(E) 视图(V) 终端(T) 标签(A) 帮助(H)

[root@master ~]# vim score.sh
[root@master ~]# chmod a+x score.sh
[root@master ~]#
```

3、运行

```
[root@master ~]# vim score.sh
[root@master ~]# chmod a+x score.sh
[root@master ~]# ./score.sh
请输入学号：20111001
学号      姓名      班级      实验课      平时成绩      期末考试
总成绩
20111001      白xx      信安1101      0.0      0.0      0
```

任务（2）

1. 使用 vim 创建一个脚本文件，并编辑代码。

```
#!/bin/bash
IS_someone=`who | grep $1`
while [ -z "$IS_someone" ]
do
    sleep 3
    IS_someone=`who | grep $1`
done
echo "watch it,$1 is online!" | wall
~
```

2.运行./someone.sh root

```
[root@master ~]# vim someone.sh
[root@master ~]# ./someone.sh root

Broadcast message from root@master (Fri Nov 3 20:24:00
watch it,root is online!
```

任务三: Linuxer 的 Shell 程序实现

(一) 客户端函数

1. Function (函数名称): **start**

Description (函数功能、性能等的描述): 启动实验环境, 初始化必要的设置, 可能会启动相关的实验程序。

Calls (被本函数调用的函数清单): 无直接调用其他函数, 可能调用其他初始化函数。

Called By (调用本函数的函数清单): 客户端主程序 `linuxer.bash` 中的菜单选项 1。

File Accessed (被访问的文件): 可能访问实验环境目录, 如 `./labs/`。

File Updated (被修改的文件): 实验环境相关的配置文件。

Input (输入参数说明, 包括每个参数的作用、取值说明及参数间关系。): 无直接输入, 依赖于环境变量或预配置。

Output (对输出参数的说明): 输出实验环境的启动状态。

Return (函数返回值的说明): 无返回值。启动操作通常是一个持续的过程, 直到实验环境被用户结束。

Others (其它说明): 可能涉及网络连接或资源分配。

2. Function (函数名称): **submit**

Description (函数功能、性能等的描述): 提交实验结果到服务器。

Calls (被本函数调用的函数清单): 可能调用文件上传相关的底层命令或库函数。

Called By (调用本函数的函数清单): `linuxer.bash` 中的菜单选项 2。

File Accessed (被访问的文件): `/lab/` 目录中的实验报告文件。

File Updated (被修改的文件): `/lab/` 目录中的实验报告文件。

Input (输入参数说明, 包括每个参数的作用、取值说明及参数间关系。): 需要提交的实验结果文件。

Output (对输出参数的说明): 输出提交结果的状态信息。

Return (函数返回值的说明): 无返回值, 通常以状态码或成功信息表示操作是否成功。

Others (其它说明): 需要确保网络连接正常。

3. Function (函数名称): **server**

Description (函数功能、性能等的描述): 连接到服务器, 进行身份验证和功能选择。

Calls (被本函数调用的函数清单): 调用底层的网络连接函数, 如 `ssh` 或其他客户端协议。

Called By (调用本函数的函数清单): `linuxer.bash` 中的菜单选项 3。

File Accessed (被访问的文件): 访问 `client.conf` 配置文件中的服务器信息。

File Updated (被修改的文件): 可能更新会话日志或连接状态。

Input (输入参数说明, 包括每个参数的作用、取值说明及参数间关系。): 用户输入的服务器地址。

Output (对输出参数的说明): 连接状态信息。

Return (函数返回值的说明): 无返回值, 通常会持续进行连接直到用户断开。

Others (其它说明): 需要网络连接。

(二) 服务器端函数说明

1. Function (函数名称): **register**

Description (函数功能、性能等的描述): 注册新的学生帐号, 验证输入的合法性并保存到数据库。

Calls (被本函数调用的函数清单): 调用系统命令 `useradd` 或其他相关函数来注册新用户。

Called By (调用本函数的函数清单): `server.bash` 中的菜单选项 1。

File Accessed (被访问的文件): `/etc/passwd`: 用于添加新用户。

File Updated (被修改的文件): `/etc/passwd`: 更新新用户信息。

Input (输入参数说明, 包括每个参数的作用、取值说明及参数间关系。): 学生的学号, 作为新用户的唯一标识符。

Output (对输出参数的说明): 输出注册结果, 包括是否成功注册了用户。

Return (函数返回值的说明): 无返回值, 注册成功则显示成功信息, 否则显示失败信息。

Others (其它说明): 需要进行输入合法性检查。

2. Function (函数名称): **checkscore**

Description (函数功能、性能等的描述): 该函数用于学生查询成绩, 通常会从服务器获取学生的实验成绩并展示给学生。

Calls (被本函数调用的函数清单): 可能调用文件读取或数据库查询函数来获取成绩。

Called By (调用本函数的函数清单): `server-students.bash` 中的菜单选项 2。

File Accessed (被访问的文件): 存储学生成绩的目录。

File Updated (被修改的文件): 无直接更新。

Input (输入参数说明, 包括每个参数的作用、取值说明及参数间关系。): 学生 ID。

Output (对输出参数的说明): 输出学生成绩信息。

Return (函数返回值的说明): 返回成绩信息。

Others (其它说明): 需要验证学生身份。

3. Function (函数名称): **listscore**

Description (函数功能、性能等的描述): 教师列出所有学生的成绩。

Calls (被本函数调用的函数清单): 可能调用数据库查询函数。

Called By (调用本函数的函数清单): `server-teachers.bash` 中的菜单选项 8。

File Accessed (被访问的文件): 存储学生成绩的目录。

File Updated (被修改的文件): 无直接更新。

Input (输入参数说明, 包括每个参数的作用、取值说明及参数间关系。): 教师 ID。

Output (对输出参数的说明): 输出所有学生的成绩列表。

Return (函数返回值的说明): 返回成绩列表。

Others (其它说明): 需要验证教师身份。

4. Function (函数名称): **mark**

Description (函数功能、性能等的描述): 教师批阅学生提交的实验报告。

Calls（被本函数调用的函数清单）:调用 `submit()` 或文件处理函数来读取学生提交的实验报告。可能调用文件读取和评分函数。

Called By（调用本函数的函数清单）:`server-teachers.bash` 中的菜单选项 9。

File Accessed（被访问的文件）:`/home/`: 存储学生提交的实验报告。

File Updated（被修改的文件）:`/opt/linuxer/score`: 更新学生的评分记录。

Input（输入参数说明，包括每个参数的作用、取值说明及参数间关系。）:学生 ID 和评分。

Output（对输出参数的说明）:输出评分结果。

Return（函数返回值的说明）:无返回值，直接更新成绩文件。

Others（其它说明）:需要确保文件可访问性。

5.Function（函数名称）:**ViewAccountMenu**

Description（函数功能、性能等的描述）:查看所有教师的账号信息。

Calls（被本函数调用的函数清单）:可能会调用数据库查询函数来获取教师账户的相关数据。

Called By（调用本函数的函数清单）:`server-teachers.bash` 中的菜单选项 1。

File Accessed（被访问的文件）:访问存储教师账号信息的文件

File Updated（被修改的文件）:无直接更新。

Input（输入参数说明，包括每个参数的作用、取值说明及参数间关系。）:无输入

Output（对输出参数的说明）:显示教师账号的列表和详细信息，如用户名、权限、状态等。

Return（函数返回值的说明）:无返回值。

Others（其它说明）:该功能通常提供给管理员查看所有教师的账户详情。

6.Function（函数名称）:**LockAccount**

Description（函数功能、性能等的描述）:用于加锁教师的账号

Calls（被本函数调用的函数清单）:可能会调用系统命令如 `passwd -l <username>` 或 `usermod -L <username>` 来加锁账户。

Called By（调用本函数的函数清单）:`server-teachers.bash` 中的菜单选项 2。

File Accessed（被访问的文件）:访问存储教师账号信息的文件

File Updated（被修改的文件）:更新存储教师账号的文件。

Input（输入参数说明，包括每个参数的作用、取值说明及参数间关系。）:教师账号

Output（对输出参数的说明）:提示账号已加锁或加锁失败的信息。

Return（函数返回值的说明）:无返回值。

Others（其它说明）:此操作是针对教师账户的管理功能，通常只有管理员可以执行此操作。

7.Function（函数名称）:**UnlockAccount**

Description（函数功能、性能等的描述）:解锁被加锁的教师账号。

Calls（被本函数调用的函数清单）:调用系统命令如 `passwd -u <username>` 或 `usermod -U <username>` 来解锁账户。

Called By（调用本函数的函数清单）:`server-teachers.bash` 中的菜单选项 3。

File Accessed（被访问的文件）:`/etc/passwd`

File Updated（被修改的文件）:`/etc/passwd`

Input（输入参数说明，包括每个参数的作用、取值说明及参数间关系。）:教师账号

Output（对输出参数的说明）:提示账号已解锁或解锁失败的信息。

Return（函数返回值的说明）:无返回值。

Others（其它说明）:此操作是针对教师账户的管理功能，通常只有管理员可以执行此操作。

8.Function（函数名称）:**DeleteAccount**

Description（函数功能、性能等的描述）:删除教师账户。

Calls（被本函数调用的函数清单）:调用系统命令如 `userdel <username>` 或直接删除相关账号的数据库条目。

Called By（调用本函数的函数清单）:`server-teachers.bash` 中的菜单选项 4。

File Accessed（被访问的文件）:`/etc/passwd`

File Updated（被修改的文件）:教师账号的家目录可能需要被删除。
Input（输入参数说明，包括每个参数的作用、取值说明及参数间关系。）:教师账号
Output（对输出参数的说明）:提示账号已删除或删除失败的信息。
Return（函数返回值的说明）:无返回值。
Others（其它说明）:此操作是针对教师账户的管理功能，通常只有管理员可以执行此操作。

9.Function（函数名称）: **ChangeAccountPasswd**

Description（函数功能、性能等的描述）:修改教师账号的密码。
Calls（被本函数调用的函数清单）:调用系统命令如 `passwd <username>` 来修改教师账号的密码。
Called By（调用本函数的函数清单）:server-teachers.bash 中的菜单选项 5。
File Accessed（被访问的文件）:/etc/passwd
File Updated（被修改的文件）:/etc/passwd
Input（输入参数说明，包括每个参数的作用、取值说明及参数间关系。）:教师账号和新密码
Output（对输出参数的说明）:提示密码修改成功或失败的信息。
Return（函数返回值的说明）:无返回值。
Others（其它说明）:新密码需要符合一定的安全策略

10.Function（函数名称）: **RegisterTeacherAccount**

Description（函数功能、性能等的描述）:新增一个教师账号。
Calls（被本函数调用的函数清单）:调用系统命令如 `useradd <username>` 来创建新账户。
Called By（调用本函数的函数清单）:server-teachers.bash 中的菜单选项 6。
File Accessed（被访问的文件）:/etc/passwd
File Updated（被修改的文件）:/etc/passwd
Input（输入参数说明，包括每个参数的作用、取值说明及参数间关系。）:教师账号和密码
Output（对输出参数的说明）:提示账号添加成功信息。
Return（函数返回值的说明）:无返回值。
Others（其它说明）:密码需要符合一定的安全策略

11.Function（函数名称）: **passwd**

Description（函数功能、性能等的描述）:当前登录的用户修改自己的密码。
Calls（被本函数调用的函数清单）:调用系统命令如 `passwd <username>` 来修改密码。
Called By（调用本函数的函数清单）:server-teachers.bash 中的菜单选项 7。
File Accessed（被访问的文件）:/etc/passwd
File Updated（被修改的文件）:/etc/passwd
Input（输入参数说明，包括每个参数的作用、取值说明及参数间关系。）:当前用户的旧密码和新密码。
Output（对输出参数的说明）:提示密码修改成功或失败的信息。
Return（函数返回值的说明）:无返回值。
Others（其它说明）:密码需要符合一定的安全策略

四、实验思考或体会

任务 1:

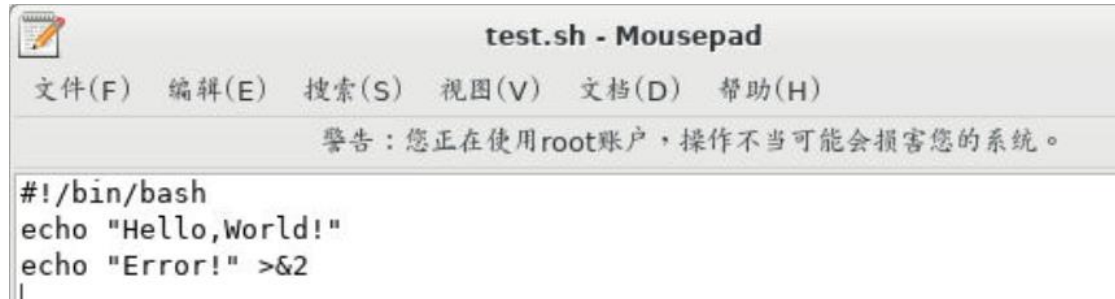
- 命令行"`command >out.file 2>&1`"是什么含义？试举例说明。
表示将 `command` 的标准输出（`stdout`）和标准错误（`stderr`）都重定向到 `out.file` 文件中，如果 `out.file` 文件已存在，其内容会被覆盖；如果不存在，会创建一个新文件。
`command` 是要执行的命令，`>` 是重定向操作符，用于将输出重定向到一个文件。`out.file` 是输出保存到的文件名。如果文件已存在，`>` 操作符会覆盖文件的内容；如果文件不存在，将创建一个新文件。`2` 表示文件描述符 2，关联着标准错误（`stderr`）。`>&1` 表示将标准错误（文

件描述符 2) 重定向到标准输出 (文件描述符 1) 的位置。&1 表示和文件描述符 1 相同的位置。

例如:

- 创建一个脚本 test.sh, 内容如下:

```
#!/bin/bash
echo "Hello,World!"
echo "Error!" >&2
```



- 赋予 test.sh 添加执行权限: `chmod +x ./test.sh`

```
[root@master Desktop]# ./test.sh > output.file
bash: ./test.sh: 权限不够
[root@master Desktop]# chmod +x ./test.sh
```

- 执行该脚本 `test.sh > output.file` (仅重定向标准输出)

```
[root@master Desktop]# ./test.sh > output.file
Error!
```



标准输出被保存在 output.file 中, 错误信息仍然显示在终端。

- 执行该脚本 `test.sh 2> output.file` (仅重定向错误输出)

```
[root@master Desktop]# ./test.sh 2> output.file
Hello,World!
```



错误输出被保存在 outfile.file 中, 标准输出仍然显示在终端。

- 执行该脚本 `test.sh > output.file 2>&1` (同时重定向标准输出和错误输出)

```
[root@master Desktop]# ./test.sh > output.file 2>&1
```

标准输出和错误输出都被保存在 output.file 中。



任务 2:

1. Shell 脚本中的程序段有错误，但该程序段在运行中没有被执行，问该错误能否被表现出来？

答：在 Shell 脚本中，如果一个程序段存在错误，但是该程序段在运行过程中没有被触发执行，那么这个错误通常不会表现出来。因为 Shell 脚本在执行时是按照从上到下的顺序解释和执行的，如果某个代码块没有被运行（比如它可能被包裹在一个条件判断语句中，而该条件判断没有为真），那么这个代码块中的错误也就不会暴露出来。

2、总结 Shell 脚本查错的基本方法和过程。

(1) 代码审查

在运行脚本之前，先进行一次代码审查，检查以下几点：

- 变量和函数的命名是否一致。
- 是否有拼写错误。
- 是否所有变量在使用前都有定义。
- 语法是否正确，特别是引号、括号是否成对出现。
- 循环和条件判断是否逻辑正确。

(2) 使用语法高亮

使用支持语法高亮的文本编辑器，这样可以帮助识别潜在的语法错误。

(3) 使用调试工具

bash -n：不执行脚本，仅检查语法错误。

bash -x：执行脚本并打印出每一行命令以及它们的执行结果，有助于跟踪脚本的执行流程。

set -u：在脚本中使用此选项，如果使用未定义的变量，脚本会报错并退出。

set -e：如果命令返回非零状态，则立即退出脚本。

(4) 添加日志

在脚本中添加 **echo** 或 **printf** 语句来输出变量的值和程序的执行状态，这样可以帮助了解脚本在执行过程中发生了什么。

(5) 单步执行

手动单步执行脚本，检查每一条命令的输出和执行结果。

(6) 分段测试

将脚本分成多个部分，逐个测试每个部分的功能，确保每部分都能正常工作。

(7) 错误处理

在脚本中添加错误处理代码，如 **trap** 命令来捕获信号和错误，以及使用条件语句来处理可能的错误情况。

(8) 使用静态分析工具

使用工具如 **shellcheck** 来分析脚本并找出潜在的错误。

(9) 查看错误日志

脚本运行出错时，检查标准错误输出和日志文件，以确定错误的具体位置和原因。